Guide complet

Batch Spring Boot pour Reprise de Données

De l'architecture à la mise en production : tout ce qu'il faut savoir pour concevoir, coder, tester, déployer et exploiter un batch de reprise de données multi-tables avec Spring Batch.

Guide technique de référence
Avril 2026

Table des matières

Partie I — Fondamentaux

1. Introduction et concepts Spring Batch
2. Architecture globale du batch
3. Structure du projet

Partie II — Mise en œuvre

4. Dépendances et configuration
5. Configuration multi-datasources
6. Writer composite et éclatement de tables
7. Gestion des rejets (SkipListener)
8. Réconciliation finale
9. Ajouter un nouveau batch pas à pas

Partie III — Performance et robustesse

10. Gestion des clés étrangères
11. Éviter les doublons et idempotence
12. Transactions et chunk size
13. Partitioning et parallélisation
14. Restart après crash

Partie IV — Production

15. Lancement du batch
16. Monitoring et observabilité
17. Gestion des secrets
18. Packaging et déploiement
19. Stratégie de rollback
20. Reprises incrémentales et bascule

Partie V — Tests et qualité

21. Stratégie de tests
22. Tests unitaires du mapper
23. Tests d'intégration avec Testcontainers
24. Tests de volumétrie

Partie VI — Références

25. Aspects réglementaires (RGPD)
26. Checklist complète
27. Erreurs fréquentes
28. Pour aller plus loin

1. Introduction et concepts Spring Batch

1.1 Qu'est-ce qu'une reprise de données

Une **reprise de données** (RDD) est une opération qui consiste à migrer, transformer ou réinjecter des données d'un système source vers un système cible. Elle intervient typiquement dans les cas suivants :

- Migration d'application (ancien SI vers nouveau)
- Changement de modèle de données (refonte de schéma BDD)
- Correction massive de données existantes
- Initialisation d'un nouveau système avec des données historiques

C'est un traitement **ponctuel ou périodique**, généralement sur de gros volumes, qu'on automatise via un batch pour garantir reproductibilité, traçabilité et performance.

> **Attention à l'acronyme :** RDD peut aussi désigner *Resilient Distributed Dataset*, la structure de base d'Apache Spark (Big Data). Dans ce document, RDD = Reprise De Données.

1.2 Pourquoi Spring Batch

Spring Batch est un framework conçu pour les traitements batch à grande échelle. Il fournit :

- **Gestion transactionnelle par chunks** : traitement par paquets avec commit intermédiaire
- **Reprise sur incident** : un job qui plante peut redémarrer où il s'est arrêté
- **Tolérance aux erreurs** : skip, retry, listeners pour tracer les rejets
- **Métadonnées persistées** : tables `BATCH_JOB_EXECUTION`, historique d'exécution
- **Parallélisation** : partitioning, multi-threading, remote chunking
- **Readers et Writers standards** : JDBC, JPA, fichiers plats, XML, JSON, Kafka

1.3 Concepts clés

Concept	Rôle
Job	Le traitement complet (ex : reprise clients). Composé d'un ou plusieurs steps.
Step	Une étape du job. De type Chunk (lecture/traitement/écriture) ou Tasklet (action unique).
ItemReader	Lit les données unitairement depuis une source.
ItemProcessor	Transforme un item : validation, mapping, enrichissement. Optionnel.
ItemWriter	Écrit les items par paquets dans une cible.
Chunk	Le paquet d'items traité dans une transaction. Ex : chunk-size=500.
Tasklet	Action unique non itérative (ex : exécuter un UPDATE SQL massif).
JobRepository	Persiste l'état d'exécution des jobs et steps en base.
JobLauncher	Déclenche l'exécution d'un Job avec des JobParameters.
ExecutionContext	Stockage clé/valeur par Job ou Step, persisté pour le restart.

2. Architecture globale du batch

2.1 Le pattern staging

Pour une RDD volumineuse où une table source s'éclate en plusieurs tables cibles, l'approche recommandée est le **pattern staging** : on intercale une couche de tables intermédiaires entre la source et les cibles.

Avantages du staging

- **Figer les données source** à un instant T : si le batch dure 4h et que la source évolue, on est protégé
- **Découpler extraction et chargement** : possibilité de relancer une phase sans l'autre
- **Tracer l'état de chaque ligne** : statut NEW / OK / KO / SKIP, motif de rejet
- **Dédoublonner et valider** via des requêtes SQL massives (rapides)
- **Rejouer facilement** en cas d'incident, sans toucher à la source

2.2 Enchaînement des steps

Le job se décompose typiquement en 5 phases :

Phase	Step(s)	Technique recommandée
1. Extraction	Source → Staging	JdbcCursorItemReader + JPA Writer
2. Validation	Marquage KO des lignes invalides	Tasklet SQL (UPDATE massif)
3. Déduplication	Marquage SKIP des doublons	Tasklet SQL
4. Chargement	Staging → tables cibles (1 step par table)	JpaItemReader/Writer
5. Réconciliation	Comptage source vs cible, rapport	Tasklet

> **Ordre d'insertion :** les steps de chargement doivent respecter l'ordre *parent avant enfants* pour préserver les contraintes de clés étrangères naturellement. Exemple : CLIENT avant COMMANDE, COMMANDE avant LIGNE_COMMANDE.

2.3 Choix JPA vs JDBC

Volume	Approche recommandée	Justification
< 100 000 lignes	JPA partout	Confort, cascades automatiques
100 000 à millions	JPA avec batch_size	Compromis perf / lisibilité
Dizaines de millions	Hybride JPA + JDBC	JDBC pour volumes, JPA pour logique
Centaines de millions	Bulk natif (COPY, SQL*Loader)	Seul moyen de tenir les délais

3. Structure du projet

3.1 Arborescence complète

```
reprise-donnees-client/
├── pom.xml
├── README.md
├── Dockerfile
├── .gitignore
│
├── docs/                                  # documentation fonctionnelle
│   ├── mapping-source-cible.md            # LE document clé en RDD
│   ├── regles-validation.md
│   ├── strategie-rejets.md
│   └── runbook-exploitation.md            # comment relancer en cas de KO
│
├── scripts/
│   ├── sql/
│   │   ├── ddl/
│   │   │   ├── V1__create_staging_tables.sql
│   │   │   ├── V2__create_rejet_tables.sql
│   │   │   └── V3__create_indexes.sql
│   │   ├── pre-batch/
│   │   │   └── desactiver_index.sql
│   │   ├── post-batch/
│   │   │   ├── reactiver_index.sql
│   │   │   └── rapport_reconciliation.sql
│   │   └── rollback/
│   │       └── vider_cibles.sql
│   └── shell/
│       ├── lancer-batch.sh
│       └── verifier-coherence.sh
│
└── src/
    ├── main/java/fr/montreprise/reprise/
    │   ├── RepriseDonneesApplication.java        # main
    │   │
    │   ├── config/                               # config transverse
    │   │   ├── BatchConfig.java
    │   │   ├── DataSourceConfig.java
    │   │   ├── JpaConfig.java
    │   │   ├── JobLauncherConfig.java
    │   │   └── AppProperties.java
    │   │
    │   ├── job/                                  # définition des JOBS
    │   │   ├── RepriseClientJobConfig.java
    │   │   └── listener/
    │   │       ├── JobExecutionListener.java
    │   │       └── JobNotificationListener.java
    │   │
    │   ├── step/                                 # STEPS par domaine
    │   │   ├── extraction/
    │   │   ├── validation/
    │   │   ├── deduplication/
    │   │   ├── chargement/
    │   │   │   ├── client/
    │   │   │   ├── adresse/
    │   │   │   ├── contact/
    │   │   │   └── compte/
    │   │   ├── reconciliation/
    │   │   └── listener/
    │   │       ├── ChunkLoggingListener.java
    │   │       ├── SkipListener.java
    │   │       └── StepMetricsListener.java
    │   │
    │   ├── domain/                               # modèle métier
    │   │   ├── legacy/                           # entités SOURCE (RO)
    │   │   ├── staging/                          # tables de staging
    │   │   ├── cible/                            # entités CIBLE
    │   │   │   ├── client/
    │   │   │   ├── adresse/
    │   │   │   └── contact/
    │   │   └── rejet/                            # traçabilité
    │   │
    │   ├── mapper/                               # transformation
    │   │   ├── ClientMapper.java
    │   │   └── AdresseMapper.java
    │   │
    │   ├── service/                              # logique réutilisable
    │   │   ├── validation/
    │   │   ├── enrichissement/
```

```
    │   │   └── rejet/
    │   │
    │   ├── rest/                                # endpoints lancement
    │   │   └── BatchController.java
    │   │
    │   └── exception/
    │       ├── RepriseException.java
    │       ├── DonneeInvalideException.java
    │       └── RepriseTechniqueException.java
    │
    ├── main/resources/
    │   ├── application.yml
    │   ├── application-dev.yml
    │   ├── application-recette.yml
    │   ├── application-prod.yml
    │   ├── logback-spring.xml
    │   └── db/migration/                        # Flyway
    │       ├── V1__schema_staging.sql
    │       ├── V2__schema_cible.sql
    │       └── V3__schema_rejet.sql
    │
    └── test/
        ├── java/fr/montreprise/reprise/
        │   ├── step/
        │   ├── service/
        │   ├── mapper/
        │   └── integration/
        │       ├── RepriseJobIT.java
        │       └── TestContainersConfig.java
        └── resources/
            └── jeu-test/
                ├── clients.csv
                └── commandes.csv
```

3.2 Rôle de chaque package

Package	Rôle
`config/`	Configuration transverse : datasources, EntityManagers, JobRepository
`job/`	Assemblage des steps dans un ordre précis
`step/`	Définitions détaillées de chaque step avec Reader/Processor/Writer
`domain/legacy/`	Entités qui mappent l'ancienne BDD, lecture seule
`domain/staging/`	Tables techniques du batch avec colonnes de pilotage
`domain/cible/`	Entités de la nouvelle BDD, celles qui resteront en production
`domain/rejet/`	Traçabilité des lignes rejetées, conservé après le batch
`mapper/`	Logique de transformation source→cible, testable sans Spring Batch
`service/`	Logique métier réutilisable : validation, enrichissement, référentiels
`rest/`	Endpoints HTTP pour déclencher/monitorer les batchs
`exception/`	Exceptions métier et techniques spécifiques à la reprise

4. Dépendances et configuration

4.1 pom.xml complet

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0">
    <modelVersion>4.0.0</modelVersion>

    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>3.3.0</version>
    </parent>

    <groupId>fr.montreprise</groupId>
    <artifactId>reprise-donnees</artifactId>
    <version>1.0.0</version>

    <properties>
        <java.version>21</java.version>
        <mapstruct.version>1.5.5.Final</mapstruct.version>
    </properties>

    <dependencies>
        <!-- Spring Batch -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-batch</artifactId>
        </dependency>

        <!-- JPA -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-jpa</artifactId>
        </dependency>

        <!-- Web (pour endpoints de lancement) -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>

        <!-- Actuator pour monitoring -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-actuator</artifactId>
        </dependency>

        <!-- Validation -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-validation</artifactId>
        </dependency>

        <!-- Drivers BDD -->
        <dependency>
            <groupId>org.postgresql</groupId>
            <artifactId>postgresql</artifactId>
        </dependency>
        <dependency>
            <groupId>com.oracle.database.jdbc</groupId>
            <artifactId>ojdbc11</artifactId>
            <version>23.3.0.23.09</version>
        </dependency>

        <!-- Flyway migrations -->
        <dependency>
            <groupId>org.flywaydb</groupId>
            <artifactId>flyway-core</artifactId>
        </dependency>
        <dependency>
            <groupId>org.flywaydb</groupId>
            <artifactId>flyway-database-postgresql</artifactId>
        </dependency>

        <!-- MapStruct -->
        <dependency>
            <groupId>org.mapstruct</groupId>
            <artifactId>mapstruct</artifactId>
            <version>${mapstruct.version}</version>
        </dependency>
```

```
        <!-- Micrometer pour métriques -->
        <dependency>
            <groupId>io.micrometer</groupId>
            <artifactId>micrometer-registry-prometheus</artifactId>
        </dependency>

        <!-- Lombok -->
        <dependency>
            <groupId>org.projectlombok</groupId>
            <artifactId>lombok</artifactId>
            <optional>true</optional>
        </dependency>

        <!-- Tests -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-test</artifactId>
            <scope>test</scope>
        </dependency>
        <dependency>
            <groupId>org.springframework.batch</groupId>
            <artifactId>spring-batch-test</artifactId>
            <scope>test</scope>
        </dependency>
        <dependency>
            <groupId>org.testcontainers</groupId>
            <artifactId>postgresql</artifactId>
            <scope>test</scope>
        </dependency>
        <dependency>
            <groupId>org.testcontainers</groupId>
            <artifactId>junit-jupiter</artifactId>
            <scope>test</scope>
        </dependency>
    </dependencies>
</project>
```

4.2 application.yml complet

```
spring:
  application:
    name: reprise-donnees

  batch:
    job:
      enabled: false                    # on déclenche manuellement
    jdbc:
      initialize-schema: never          # géré par Flyway
      table-prefix: BATCH_

  jpa:
    open-in-view: false
    hibernate:
      ddl-auto: validate
    properties:
      hibernate:
        jdbc:
          batch_size: 500
          fetch_size: 1000
        order_inserts: true
        order_updates: true
        generate_statistics: false
        dialect: org.hibernate.dialect.PostgreSQLDialect

  flyway:
    enabled: true
    locations: classpath:db/migration
    baseline-on-migrate: true

server:
  port: 8080

# Datasources
app:
  datasource:
    source:
      jdbc-url: jdbc:oracle:thin:@legacy-db:1521/XE
      username: ${SOURCE_USER}
      password: ${SOURCE_PWD}
      driver-class-name: oracle.jdbc.OracleDriver
      hikari:
        maximum-pool-size: 5
```

```yaml
        read-only: true

    cible:
      jdbc-url: jdbc:postgresql://new-db:5432/app
      username: ${CIBLE_USER}
      password: ${CIBLE_PWD}
      driver-class-name: org.postgresql.Driver
      hikari:
        maximum-pool-size: 20

    batch:
      jdbc-url: jdbc:postgresql://batch-db:5432/spring_batch
      username: ${BATCH_USER}
      password: ${BATCH_PWD}
      hikari:
        maximum-pool-size: 5

  # Configuration métier
  batch:
    chunk-size:
      extraction: 1000
      validation: 5000
      chargement-client: 500
      chargement-adresse: 500
      chargement-contact: 1000
    skip-limit: 1000
    notification:
      email-ko: equipe-exploit@entreprise.fr
      slack-webhook: ${SLACK_WEBHOOK:}

# Actuator / monitoring
management:
  endpoints:
    web:
      exposure:
        include: health,info,metrics,prometheus,batch
  metrics:
    export:
      prometheus:
        enabled: true

logging:
  level:
    root: INFO
    fr.montreprise.reprise: DEBUG
    org.springframework.batch: INFO
    org.hibernate.SQL: WARN
```

5. Configuration multi-datasources

5.1 Pourquoi trois datasources

Une RDD utilise quasi systématiquement trois bases distinctes :

- **Source** : l'ancienne BDD en lecture seule
- **Cible** : la nouvelle BDD où on écrit les données migrées
- **Batch** : les métadonnées Spring Batch (BATCH_JOB_EXECUTION, etc.)

Séparer la base batch évite de polluer la base cible avec des tables techniques, et permet de garder l'historique d'exécution même si la base cible est reset.

5.2 DataSourceConfig.java

```
@Configuration
public class DataSourceConfig {

    @Bean(name = "sourceDataSource")
    @ConfigurationProperties("app.datasource.source")
    public DataSource sourceDataSource() {
        return DataSourceBuilder.create().type(HikariDataSource.class).build();
    }

    @Bean(name = "cibleDataSource")
    @Primary
    @ConfigurationProperties("app.datasource.cible")
    public DataSource cibleDataSource() {
        return DataSourceBuilder.create().type(HikariDataSource.class).build();
    }

    @Bean(name = "batchDataSource")
    @BatchDataSource
    @ConfigurationProperties("app.datasource.batch")
    public DataSource batchDataSource() {
        return DataSourceBuilder.create().type(HikariDataSource.class).build();
    }

    @Bean(name = "cibleTransactionManager")
    @Primary
    public PlatformTransactionManager cibleTransactionManager(
            @Qualifier("cibleEntityManagerFactory") EntityManagerFactory emf) {
        return new JpaTransactionManager(emf);
    }

    @Bean(name = "batchTransactionManager")
    public PlatformTransactionManager batchTransactionManager(
            @Qualifier("batchDataSource") DataSource ds) {
        return new DataSourceTransactionManager(ds);
    }
}
```

5.3 JpaConfig.java — EntityManager par datasource

```
@Configuration
@EnableJpaRepositories(
    basePackages = "fr.montreprise.reprise.domain.cible",
    entityManagerFactoryRef = "cibleEntityManagerFactory",
    transactionManagerRef = "cibleTransactionManager"
)
public class CibleJpaConfig {

    @Bean(name = "cibleEntityManagerFactory")
    @Primary
    public LocalContainerEntityManagerFactoryBean cibleEntityManagerFactory(
            @Qualifier("cibleDataSource") DataSource dataSource,
            EntityManagerFactoryBuilder builder) {
        return builder
            .dataSource(dataSource)
            .packages("fr.montreprise.reprise.domain.cible",
                      "fr.montreprise.reprise.domain.staging",
                      "fr.montreprise.reprise.domain.rejet")
            .persistenceUnit("cible")
            .build();
    }
}

@Configuration
@EnableJpaRepositories(
    basePackages = "fr.montreprise.reprise.domain.legacy",
    entityManagerFactoryRef = "sourceEntityManagerFactory",
    transactionManagerRef = "sourceTransactionManager"
)
public class SourceJpaConfig {

    @Bean(name = "sourceEntityManagerFactory")
    public LocalContainerEntityManagerFactoryBean sourceEntityManagerFactory(
            @Qualifier("sourceDataSource") DataSource dataSource,
            EntityManagerFactoryBuilder builder) {
        return builder
            .dataSource(dataSource)
            .packages("fr.montreprise.reprise.domain.legacy")
            .persistenceUnit("source")
            .properties(Map.of("hibernate.dialect",
                               "org.hibernate.dialect.OracleDialect"))
            .build();
    }

    @Bean(name = "sourceTransactionManager")
    public PlatformTransactionManager sourceTransactionManager(
            @Qualifier("sourceEntityManagerFactory") EntityManagerFactory emf) {
        return new JpaTransactionManager(emf);
    }
}
```

> **Attention :** chaque `@EnableJpaRepositories` doit pointer vers un `basePackages` différent, sinon Spring ne sait pas quel repository rattacher à quel EntityManager.

6. Writer composite et éclatement de tables

6.1 Le problème

Quand une ligne source doit alimenter plusieurs tables cibles, deux approches sont possibles :

Approche	Quand l'utiliser
JPA + cascades	Les entités cibles sont liées par des relations `@OneToMany`, `@OneToOne`
Writer composite	Les tables cibles sont indépendantes ou reliées par clé métier, ou on veut du contrôle fin

6.2 Solution JPA avec cascades

```
@Entity
@Table(name = "commande")
public class Commande {
    @Id @GeneratedValue private Long id;
    @Column(unique = true) private Long idExterne;

    @OneToMany(mappedBy = "commande",
               cascade = CascadeType.ALL,
               orphanRemoval = true)
    private List<LigneCommande> lignes = new ArrayList<>();

    @OneToOne(mappedBy = "commande", cascade = CascadeType.ALL)
    private Livraison livraison;
}

// Le JpaItemWriter écrit Commande, Lignes et Livraison en une seule opération
@Bean
public JpaItemWriter<Commande> commandeWriter(EntityManagerFactory emf) {
    JpaItemWriter<Commande> writer = new JpaItemWriter<>();
    writer.setEntityManagerFactory(emf);
    return writer;
}
```

6.3 Solution Writer composite personnalisé

Quand tu veux plus de contrôle ou que les tables ne sont pas liées par JPA :

```
@Data @AllArgsConstructor
public class ClientAggregate {
    private Client client;
    private Adresse adresse;
    private List<Contact> contacts;
    private CompteBancaire compteBancaire;
}

@Bean
public ItemWriter<ClientAggregate> compositeWriter(
        JdbcBatchItemWriter<Client> clientWriter,
        JdbcBatchItemWriter<Adresse> adresseWriter,
        JdbcBatchItemWriter<Contact> contactWriter,
        JdbcBatchItemWriter<CompteBancaire> compteWriter) {

    return aggregates -> {
        // Extraire chaque type vers sa liste
        List<Client> clients = aggregates.getItems().stream()
                .map(ClientAggregate::getClient).toList();

        List<Adresse> adresses = aggregates.getItems().stream()
                .map(ClientAggregate::getAdresse).toList();

        // flatMap car 1 agrégat → plusieurs contacts
        List<Contact> contacts = aggregates.getItems().stream()
                .flatMap(a -> a.getContacts().stream()).toList();

        List<CompteBancaire> comptes = aggregates.getItems().stream()
                .map(ClientAggregate::getCompteBancaire)
                .filter(Objects::nonNull).toList();

        // Ordre IMPORTANT : respecter les FK (parent avant enfants)
        clientWriter.write(new Chunk<>(clients));
        if (!adresses.isEmpty()) adresseWriter.write(new Chunk<>(adresses));
        if (!contacts.isEmpty()) contactWriter.write(new Chunk<>(contacts));
        if (!comptes.isEmpty()) compteWriter.write(new Chunk<>(comptes));
    };
}

@Bean
public JdbcBatchItemWriter<Client> clientWriter(
        @Qualifier("cibleDataSource") DataSource ds) {
    return new JdbcBatchItemWriterBuilder<Client>()
        .dataSource(ds)
        .sql("""
             INSERT INTO client (id_externe, nom, prenom, email)
             VALUES (:idExterne, :nom, :prenom, :email)
             ON CONFLICT (id_externe) DO UPDATE SET
                 nom = EXCLUDED.nom,
                 prenom = EXCLUDED.prenom,
                 email = EXCLUDED.email
             """)
        .beanMapped()
        .build();
}
```

> **Cardinalité 1→N :** une ligne source peut produire 0, 1 ou N lignes cibles. Le `flatMap` gère ça mais attention au chunk size : si chaque client a 5 contacts, un chunk de 1000 clients = 5000 inserts contacts.

7. Gestion des rejets (SkipListener)

7.1 Modéliser les rejets

Créer une table dédiée pour tracer tous les rejets :

```
CREATE TABLE rejet_reprise (
    id BIGSERIAL PRIMARY KEY,
    job_execution_id BIGINT NOT NULL,
    step_name VARCHAR(100) NOT NULL,
    type_entite VARCHAR(50) NOT NULL,          -- CLIENT, COMMANDE...
    id_source BIGINT,                          -- clé de la ligne source
    phase VARCHAR(20),                         -- READ, PROCESS, WRITE
    exception_class VARCHAR(200),
    message VARCHAR(1000),
    payload TEXT,                              -- ligne source sérialisée (JSON)
    date_rejet TIMESTAMP NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_rejet_job ON rejet_reprise(job_execution_id);
CREATE INDEX idx_rejet_type ON rejet_reprise(type_entite);
```

7.2 Entité et repository

```
@Entity
@Table(name = "rejet_reprise")
@Getter @Setter @NoArgsConstructor
public class Rejet {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private Long jobExecutionId;
    private String stepName;
    private String typeEntite;
    private Long idSource;
    private String phase;
    private String exceptionClass;

    @Column(length = 1000)
    private String message;

    @Column(columnDefinition = "TEXT")
    private String payload;

    private LocalDateTime dateRejet = LocalDateTime.now();
}

public interface RejetRepository extends JpaRepository<Rejet, Long> {
    long countByJobExecutionIdAndTypeEntite(Long jobId, String type);
    List<Rejet> findByJobExecutionIdAndTypeEntite(Long jobId, String type);
}
```

7.3 Le SkipListener

```
@Component
@StepScope
@RequiredArgsConstructor
public class RejetSkipListener<I, O> implements SkipListener<I, O> {

    private final RejetRepository rejetRepository;
    private final ObjectMapper objectMapper;

    @Value("#{stepExecution.jobExecutionId}")
    private Long jobExecutionId;

    @Value("#{stepExecution.stepName}")
    private String stepName;

    @Override
    public void onSkipInRead(Throwable t) {
        saveRejet(null, "READ", t, null);
    }

    @Override
    public void onSkipInProcess(I item, Throwable t) {
        saveRejet(extractId(item), "PROCESS", t, item);
    }

    @Override
    public void onSkipInWrite(O item, Throwable t) {
        saveRejet(extractId(item), "WRITE", t, item);
    }

    private void saveRejet(Long idSource, String phase, Throwable t, Object payload) {
        try {
            Rejet rejet = new Rejet();
            rejet.setJobExecutionId(jobExecutionId);
            rejet.setStepName(stepName);
            rejet.setTypeEntite(deduireTypeEntite(stepName));
            rejet.setIdSource(idSource);
            rejet.setPhase(phase);
            rejet.setExceptionClass(t.getClass().getName());
            rejet.setMessage(t.getMessage());
            if (payload != null) {
                rejet.setPayload(objectMapper.writeValueAsString(payload));
            }
            rejetRepository.save(rejet);
        } catch (Exception e) {
            // Ne jamais faire planter le batch parce que le log de rejet a raté
            log.error("Impossible de sauvegarder le rejet", e);
        }
    }

    private Long extractId(Object item) {
        if (item instanceof StgClient s) return s.getIdLegacy();
        if (item instanceof StgCommande c) return c.getIdCmd();
        return null;
    }

    private String deduireTypeEntite(String stepName) {
        if (stepName.contains("Client")) return "CLIENT";
        if (stepName.contains("Commande")) return "COMMANDE";
        return "INCONNU";
    }
}
```

7.4 Brancher le listener sur le step

```
@Bean
public Step chargementClientStep(
        RejetSkipListener<StgClient, Client> skipListener,
        // ... autres dépendances
) {
    return new StepBuilder("chargementClientStep", jobRepository)
            .<StgClient, Client>chunk(500, transactionManager)
            .reader(reader)
            .processor(processor)
            .writer(writer)
            .faultTolerant()
            .skip(DonneeInvalideException.class)
            .skip(DataIntegrityViolationException.class)
            .skipLimit(1000)
            .listener(skipListener)
            .build();
}
```

8. Réconciliation finale

8.1 Objectif

À la fin du batch, un step de réconciliation compte les lignes côté source, côté cible, côté rejets, et vérifie la cohérence. Il alerte si l'écart dépasse un seuil.

8.2 Tasklet de réconciliation

```
@Component
@RequiredArgsConstructor
@Slf4j
public class ReconciliationTasklet implements Tasklet {

    private final JdbcTemplate jdbcTemplateSource;
    private final JdbcTemplate jdbcTemplateCible;
    private final RejetRepository rejetRepository;
    private final NotificationService notifService;

    private static final double SEUIL_ALERTE_POURCENT = 1.0;  // 1%

    @Override
    public RepeatStatus execute(StepContribution contribution,
                                ChunkContext context) {

        Long jobId = context.getStepContext()
                            .getStepExecution()
                            .getJobExecutionId();

        // Compter côté source (ceux à migrer)
        Long nbSource = jdbcTemplateSource.queryForObject(
            "SELECT COUNT(*) FROM client_legacy WHERE migre = false",
            Long.class);

        // Compter côté cible (ceux créés par ce batch)
        Long nbCible = jdbcTemplateCible.queryForObject(
            "SELECT COUNT(*) FROM client WHERE id_externe IN " +
            "(SELECT id_legacy FROM stg_client WHERE statut_traitement = 'OK')",
            Long.class);

        // Compter les rejets
        long nbRejetsClient = rejetRepository
            .countByJobExecutionIdAndTypeEntite(jobId, "CLIENT");

        // Compter les skip volontaires
        Long nbSkip = jdbcTemplateCible.queryForObject(
            "SELECT COUNT(*) FROM stg_client WHERE statut_traitement = 'SKIP_DOUBLON'",
            Long.class);

        long manquants = nbSource - nbCible - nbRejetsClient - nbSkip;

        // Rapport
        RapportReconciliation rapport = new RapportReconciliation(
            nbSource, nbCible, nbRejetsClient, nbSkip, manquants);

        log.info("Réconciliation : {}", rapport);

        // Stockage du rapport dans l'ExecutionContext pour consultation
        context.getStepContext().getStepExecution()
               .getJobExecution().getExecutionContext()
               .put("rapportReconciliation", rapport);

        // Alerte si écart trop grand
        double ecartPourcent = (Math.abs(manquants) * 100.0) / nbSource;
        if (ecartPourcent > SEUIL_ALERTE_POURCENT) {
            String msg = String.format(
                "ÉCART DE RÉCONCILIATION : %d lignes manquantes (%.2f%%)",
                manquants, ecartPourcent);
            log.error(msg);
            notifService.envoyerAlerte("Reprise KO", msg);
            contribution.setExitStatus(
                new ExitStatus("ECART_RECONCILIATION", msg));
        }

        return RepeatStatus.FINISHED;
    }
}
```

9. Ajouter un nouveau batch pas à pas

Exemple : ajouter la reprise de la table `COMMANDE_LEGACY` qui s'éclate en `COMMANDE`, `LIGNE_COMMANDE` et `LIVRAISON`.

Étape 1 — Documenter le mapping

Dans `docs/mapping-source-cible.md`, un tableau validé par la MOA :

Colonne source	Table cible	Colonne cible	Règle
id_cmd	commande	id_externe	Clé de rapprochement
montant_ht	commande	montant_ht	Rejet si < 0
ref_prod_1..5	ligne_commande	reference_produit	1 ligne par ref non null
adresse_livraison	livraison	adresse	Normalisation

Étape 2 — Entité legacy

```
@Entity
@Table(name = "commande_legacy")
@Immutable
@Getter
public class CommandeLegacy {
    @Id @Column(name = "id_cmd") private Long idCmd;
    @Column(name = "id_client") private Long idClient;
    @Column(name = "date_cmd") private LocalDateTime dateCmd;
    @Column(name = "montant_ht") private BigDecimal montantHt;
    @Column(name = "ref_prod_1") private String refProd1;
    @Column(name = "qte_prod_1") private Integer qteProd1;
    // ... prod_2 à prod_5
    @Column(name = "adresse_livraison") private String adresseLivraison;
}
```

Étape 3 — Migration Flyway staging

```
-- V4__staging_commande.sql
CREATE TABLE stg_commande (
    id_cmd BIGINT PRIMARY KEY,
    id_client BIGINT,
    date_cmd TIMESTAMP,
    montant_ht DECIMAL(12,2),
    ref_prod_1 VARCHAR(50), qte_prod_1 INTEGER,
    ref_prod_2 VARCHAR(50), qte_prod_2 INTEGER,
    ref_prod_3 VARCHAR(50), qte_prod_3 INTEGER,
    ref_prod_4 VARCHAR(50), qte_prod_4 INTEGER,
    ref_prod_5 VARCHAR(50), qte_prod_5 INTEGER,
    adresse_livraison VARCHAR(500),
    statut_traitement VARCHAR(20) NOT NULL DEFAULT 'NEW',
    motif_rejet VARCHAR(500),
    date_extraction TIMESTAMP NOT NULL DEFAULT NOW(),
    hash_ligne VARCHAR(64)
);
CREATE INDEX idx_stg_commande_statut ON stg_commande(statut_traitement);
```

Étape 4 — Entités cibles

```
@Entity @Table(name = "commande")
@Getter @Setter
public class Commande {
    @Id @GeneratedValue private Long id;
    @Column(unique = true) private Long idExterne;
    @ManyToOne @JoinColumn(name = "id_client") private Client client;
    private BigDecimal montantHt;

    @OneToMany(mappedBy = "commande", cascade = CascadeType.ALL)
    private List<LigneCommande> lignes = new ArrayList<>();

    @OneToOne(mappedBy = "commande", cascade = CascadeType.ALL)
    private Livraison livraison;
}
```

Étape 5 — Mapper

```
@Component
@RequiredArgsConstructor
public class CommandeMapper {

    private final ClientRepository clientRepository;

    public Commande toCommande(StgCommande stg) {
        Commande cmd = new Commande();
        cmd.setIdExterne(stg.getIdCmd());
        cmd.setMontantHt(stg.getMontantHt());

        Client client = clientRepository.findByIdExterne(stg.getIdClient())
            .orElseThrow(() -> new DonneeInvalideException(
                "Client inexistant: " + stg.getIdClient()));
        cmd.setClient(client);

        cmd.setLignes(extraireLignes(stg, cmd));
        if (stg.getAdresseLivraison() != null) {
            cmd.setLivraison(construireLivraison(stg, cmd));
        }
        return cmd;
    }

    private List<LigneCommande> extraireLignes(StgCommande stg, Commande cmd) {
        List<LigneCommande> lignes = new ArrayList<>();
        ajouterSiPresent(lignes, cmd, stg.getRefProd1(), stg.getQteProd1());
        ajouterSiPresent(lignes, cmd, stg.getRefProd2(), stg.getQteProd2());
        ajouterSiPresent(lignes, cmd, stg.getRefProd3(), stg.getQteProd3());
        ajouterSiPresent(lignes, cmd, stg.getRefProd4(), stg.getQteProd4());
        ajouterSiPresent(lignes, cmd, stg.getRefProd5(), stg.getQteProd5());
        return lignes;
    }

    private void ajouterSiPresent(List<LigneCommande> lignes, Commande cmd,
                                  String ref, Integer qte) {
        if (ref != null && !ref.isBlank() && qte != null && qte > 0) {
            LigneCommande l = new LigneCommande();
            l.setCommande(cmd);
            l.setReferenceProduit(ref.trim().toUpperCase());
            l.setQuantite(qte);
            lignes.add(l);
        }
    }
}
```

Étape 6 — Steps (extraction, validation, chargement)

Créer les fichiers dans `step/extraction/`, `step/validation/` et `step/chargement/commande/`. Structure identique à celle détaillée en sections 6 et 7 de ce guide.

Étape 7 — Ajouter au Job

> **Ordre FK critique :** un `Commande` référence un `Client`, donc `chargementClientStep` doit être terminé avant `chargementCommandeStep`.

```
@Bean
public Job repriseGlobaleJob(
        Step extractionClientStep,
        Step validationClientStep,
        Step chargementClientStep,
        Step extractionCommandeStep,
        Step validationCommandeStep,
        Step chargementCommandeStep,
        Step reconciliationStep,
        JobExecutionListener jobListener) {

    return new JobBuilder("repriseGlobaleJob", jobRepository)
        .incrementer(new RunIdIncrementer())
        .listener(jobListener)
        .start(extractionClientStep)
        .next(validationClientStep)
        .next(chargementClientStep)         // parents d'abord
        .next(extractionCommandeStep)
        .next(validationCommandeStep)
        .next(chargementCommandeStep)       // enfants ensuite
        .next(reconciliationStep)
        .build();
}
```

Étape 8 — Tests et doc

- Tests unitaires du `CommandeMapper` (cas nominaux, limites, rejets)
- Test d'intégration du step avec Testcontainers
- Mise à jour du runbook d'exploitation

10. Gestion des clés étrangères

10.1 Règle générale

Si tu respectes l'ordre **parent → enfants** dans tes steps, les FK ne te gênent pas. **Réponse courte : ne pas désactiver les FK dans 90% des cas.**

10.2 Quand désactiver les FK

- Volumes très importants (dizaines de millions de lignes) : gain de 20-40%
- Ordre d'insertion impossible à garantir
- Données circulaires (A référence B qui référence A)

10.3 Comment désactiver proprement

PostgreSQL :

```
-- Avant le batch
ALTER TABLE adresse DROP CONSTRAINT fk_adresse_client;

-- Après le batch : réactivation AVEC validation (scanne tout)
ALTER TABLE adresse ADD CONSTRAINT fk_adresse_client
    FOREIGN KEY (id_client) REFERENCES client(id);
```

Oracle :

```
-- Désactivation
ALTER TABLE adresse DISABLE CONSTRAINT fk_adresse_client;

-- Réactivation avec validation
ALTER TABLE adresse ENABLE VALIDATE CONSTRAINT fk_adresse_client;
```

10.4 Tasklet de désactivation

```
@Bean
public Step stepDesactiverFK() {
    return new StepBuilder("desactiverFK", jobRepository)
        .tasklet((contribution, context) -> {
            jdbcTemplate.execute(
                "ALTER TABLE adresse DISABLE CONSTRAINT fk_adresse_client");
            jdbcTemplate.execute(
                "ALTER TABLE contact DISABLE CONSTRAINT fk_contact_client");
            return RepeatStatus.FINISHED;
        }, transactionManager)
        .build();
}
```

> **Attention :** désactiver les FK fait perdre l'intégrité pendant le batch. En cas de plantage, on peut se retrouver avec des orphelins. Toujours réactiver avec validation à la fin pour détecter les incohérences.

10.5 Alternatives plus efficaces

- Désactiver les **index secondaires** (pas PK/FK) pendant le chargement
- Tables en **UNLOGGED** (PostgreSQL) ou **NOLOGGING** (Oracle)
- Augmenter le **chunk size** à 1000-5000
- **Paralléliser** les steps non dépendants
- **Bulk insert natif** (`COPY` , `SQL*Loader`)

11. Éviter les doublons et idempotence

11.1 Pourquoi l'idempotence est critique

Un batch de RDD doit pouvoir être rejoué sans effet de bord : c'est la condition pour pouvoir reprendre après un crash, relancer après un bug correctif, ou simplement tester sereinement.

11.2 Stratégie 1 : clé métier stable

```
CREATE TABLE client (
    id BIGSERIAL PRIMARY KEY,
    id_externe BIGINT NOT NULL UNIQUE,   -- contrainte UNIQUE
    nom VARCHAR(100)
);
```

11.3 Stratégie 2 : UPSERT (recommandée)

PostgreSQL :

```
INSERT INTO client (id_externe, nom, prenom, email)
VALUES (:idExterne, :nom, :prenom, :email)
ON CONFLICT (id_externe) DO UPDATE SET
    nom = EXCLUDED.nom,
    prenom = EXCLUDED.prenom,
    email = EXCLUDED.email,
    date_maj = NOW();
```

Oracle :

```
MERGE INTO client c
USING (SELECT :idExterne AS id_externe FROM dual) src
ON (c.id_externe = src.id_externe)
WHEN MATCHED THEN UPDATE SET nom = :nom
WHEN NOT MATCHED THEN INSERT (id_externe, nom) VALUES (:idExterne, :nom);
```

11.4 Stratégie 3 : pré-check en mémoire

Viable jusqu'à quelques millions de clés (HashSet<Long> de 10M ≈ 400 Mo).

```
@Bean
@StepScope
public ItemProcessor<ClientLegacy, Client> clientProcessor() {
    Set<Long> existants = new HashSet<>(
        jdbcTemplate.queryForList("SELECT id_externe FROM client", Long.class)
    );
    return legacy -> existants.contains(legacy.getId())
        ? null      // skip, déjà migré
        : mapToClient(legacy);
}
```

11.5 Stratégie 4 : dédoublonnage dans le staging

```
UPDATE stg_client_legacy
SET statut_traitement = 'SKIP_DOUBLON',
    motif_rejet = 'Email en doublon, ligne plus récente conservée'
WHERE id NOT IN (
    SELECT MAX(id) FROM stg_client_legacy
    GROUP BY LOWER(TRIM(email))
);
```

11.6 Stratégie 5 : hash de ligne pour les deltas

```
ALTER TABLE stg_client_legacy ADD COLUMN hash_ligne VARCHAR(64);

UPDATE stg_client_legacy
SET hash_ligne = MD5(COALESCE(nom,'') || COALESCE(prenom,'')
                   || COALESCE(email,''));
```

Si le hash n'a pas changé depuis la dernière exécution, la ligne est sautée. Gros gain sur les reprises incrémentales.

12. Transactions et chunk size

12.1 Comprendre les transactions Spring Batch

Chaque chunk s'exécute dans **une transaction unique**. Si une écriture plante, tout le chunk rollback. Le chunk suivant démarre dans une nouvelle transaction.

Chunk size	Avantages	Inconvénients
Petit (50-100)	Rollbacks légers, reprise précise	Beaucoup de commits = lent
Moyen (500-1000)	Bon compromis perf/robustesse	-
Grand (5000+)	Moins de commits = rapide	OOM possible, rollback coûteux

12.2 Configuration fine des transactions

```
@Bean
public Step monStep() {
    return new StepBuilder("monStep", jobRepository)
        .<Source, Cible>chunk(1000, transactionManager)
        .reader(reader)
        .processor(processor)
        .writer(writer)
        // Attributs transactionnels fins
        .transactionAttribute(attributs())
        .build();
}

private TransactionAttribute attributs() {
    DefaultTransactionAttribute attr = new DefaultTransactionAttribute();
    attr.setPropagationBehavior(Propagation.REQUIRED.value());
    attr.setIsolationLevel(Isolation.READ_COMMITTED.value());
    attr.setTimeout(300);      // 5 minutes max par chunk
    return attr;
}
```

12.3 Skip et retry

```
@Bean
public Step stepRobuste() {
    return new StepBuilder("stepRobuste", jobRepository)
        .<I, O>chunk(500, tx)
        .reader(reader).processor(processor).writer(writer)
        .faultTolerant()
        // SKIP : passe 1'item fautif et continue
        .skip(DonneeInvalideException.class)
        .skipLimit(1000)
        // RETRY : réessaie avant de skip (pour erreurs transitoires)
        .retry(DeadlockLoserDataAccessException.class)
        .retryLimit(3)
        .backOffPolicy(new ExponentialBackOffPolicy())
        .build();
}
```

13. Partitioning et parallélisation

13.1 Quand partitionner

- Volumes énormes (10M+ lignes) avec fenêtre batch serrée
- BDD capable d'encaisser plusieurs connexions en parallèle
- Pas de dépendance d'ordre entre les chunks

13.2 Partitioning local (multi-thread)

```
@Bean
public Step partitionedStep(Step workerStep,
                            Partitioner partitioner,
                            TaskExecutor taskExecutor) {
    return new StepBuilder("partitionedStep", jobRepository)
        .partitioner("workerStep", partitioner)
        .step(workerStep)
        .gridSize(8)                        // 8 partitions
        .taskExecutor(taskExecutor)
        .build();
}

@Bean
public Partitioner rangePartitioner() {
    return gridSize -> {
        Long min = jdbcTemplate.queryForObject(
            "SELECT MIN(id) FROM stg_client WHERE statut='OK'", Long.class);
        Long max = jdbcTemplate.queryForObject(
            "SELECT MAX(id) FROM stg_client WHERE statut='OK'", Long.class);

        long range = (max - min) / gridSize + 1;
        Map<String, ExecutionContext> partitions = new HashMap<>();

        for (int i = 0; i < gridSize; i++) {
            ExecutionContext ctx = new ExecutionContext();
            ctx.putLong("minId", min + i * range);
            ctx.putLong("maxId", Math.min(min + (i+1) * range - 1, max));
            partitions.put("partition" + i, ctx);
        }
        return partitions;
    };
}

@Bean
public TaskExecutor taskExecutor() {
    ThreadPoolTaskExecutor exec = new ThreadPoolTaskExecutor();
    exec.setCorePoolSize(8);
    exec.setMaxPoolSize(8);
    exec.setQueueCapacity(16);
    exec.setThreadNamePrefix("batch-worker-");
    exec.initialize();
    return exec;
}

// Le reader du worker step lit sa propre plage
@Bean
@StepScope
public JdbcCursorItemReader<StgClient> partitionReader(
        @Value("#{stepExecutionContext['minId']}") Long minId,
        @Value("#{stepExecutionContext['maxId']}") Long maxId,
        @Qualifier("cibleDataSource") DataSource ds) {

    return new JdbcCursorItemReaderBuilder<StgClient>()
        .name("partitionReader")
        .dataSource(ds)
        .sql("SELECT * FROM stg_client WHERE id BETWEEN ? AND ? " +
             "AND statut_traitement = 'OK'")
        .preparedStatementSetter((ps) -> {
            ps.setLong(1, minId);
            ps.setLong(2, maxId);
        })
        .rowMapper(new StgClientRowMapper())
        .build();
}
```

> **Attention au pool de connexions :** si tu as 8 threads, ton pool Hikari doit avoir au moins 10 connexions (8 workers + marge).

14. Restart après crash

14.1 Le restart natif Spring Batch

Spring Batch est conçu pour être redémarrable. Après un crash, il reprend au début du step qui a échoué (ou avant le dernier chunk commité selon le type de reader).

14.2 Conditions pour que le restart fonctionne

- **JobRepository persistant** (pas en mémoire) : les métadonnées survivent au crash
- **Beans avec** `@StepScope` : les readers et processors doivent être recréés à chaque step pour reprendre proprement leur état
- **Readers sauvegardent leur état** : `JdbcCursorItemReader` et `JpaPagingItemReader` gèrent ça nativement
- **Même** `jobParameters` : le restart demande les mêmes paramètres que l'exécution qui a planté

14.3 Lancer un restart

```
@Service
@RequiredArgsConstructor
public class BatchRestartService {

    private final JobOperator jobOperator;
    private final JobExplorer jobExplorer;

    public Long restart(Long jobExecutionIdEchouee)
            throws JobInstanceAlreadyCompleteException {
        // Le JobOperator relit les paramètres de l'exec échouée
        // et relance avec les mêmes
        return jobOperator.restart(jobExecutionIdEchouee);
    }

    public List<JobExecution> executionsFailed(String jobName) {
        Set<JobExecution> all = jobExplorer
            .findRunningJobExecutions(jobName);
        return all.stream()
            .filter(e -> e.getStatus() == BatchStatus.FAILED)
            .toList();
    }
}
```

14.4 Step non redémarrable

Certains steps ne doivent pas être redémarrés (ex : envoi de mail, appel API non idempotent). On peut les marquer :

```
@Bean
public Step stepNonRestartable() {
    return new StepBuilder("stepNonRestartable", jobRepository)
        .tasklet(maTasklet(), tx)
        .allowStartIfComplete(false)        // interdit le rerun si déjà OK
        .startLimit(1)                       // max 1 exécution
        .build();
}
```

15. Lancement du batch

15.1 Les trois modes de lancement

Mode	Quand	Pros/Cons
Ligne de commande	Opérationnel, CI/CD	+ simple, - pas de UI
Endpoint REST	Déclenchement manuel, interface admin	+ UI possible, - sécurité à gérer
Ordonnanceur (cron, Control-M, Airflow)	Production planifiée	+ suivi pro, - infra à prévoir

15.2 Ligne de commande

```
# application.yml : spring.batch.job.enabled=false
# → on passe le nom du job en argument

java -jar reprise.jar \
  --spring.profiles.active=prod \
  --spring.batch.job.name=repriseGlobaleJob \
  --dateReprise=2026-04-20

# Avec variables d'env pour les secrets
export SOURCE_USER=legacy
export SOURCE_PWD=***
java -jar reprise.jar ...
```

15.3 Endpoint REST

```
@RestController
@RequestMapping("/api/batch")
@RequiredArgsConstructor
public class BatchController {

    private final JobLauncher jobLauncher;
    private final JobExplorer jobExplorer;
    private final Job repriseGlobaleJob;

    @PostMapping("/reprise/lancer")
    @PreAuthorize("hasRole('BATCH_ADMIN')")
    public ResponseEntity<LancementResponse> lancer(
            @RequestBody LancementRequest request) throws Exception {

        JobParameters params = new JobParametersBuilder()
            .addLong("timestamp", System.currentTimeMillis())
            .addString("demandeur", request.demandeur())
            .addString("modeTest", String.valueOf(request.modeTest()))
            .toJobParameters();

        JobExecution exec = jobLauncher.run(repriseGlobaleJob, params);

        return ResponseEntity.accepted().body(new LancementResponse(
            exec.getId(), exec.getStatus().toString()));
    }

    @GetMapping("/executions/{id}")
    public ResponseEntity<JobStatusDto> status(@PathVariable Long id) {
        JobExecution exec = jobExplorer.getJobExecution(id);
        if (exec == null) return ResponseEntity.notFound().build();

        return ResponseEntity.ok(new JobStatusDto(
            exec.getId(),
            exec.getStatus().toString(),
            exec.getStartTime(),
            exec.getEndTime(),
            exec.getStepExecutions().stream()
                .map(s -> new StepStatusDto(
                    s.getStepName(),
                    s.getStatus().toString(),
                    s.getReadCount(),
                    s.getWriteCount(),
                    s.getSkipCount()))
                .toList()));
    }

    @PostMapping("/executions/{id}/stop")
    @PreAuthorize("hasRole('BATCH_ADMIN')")
    public ResponseEntity<Void> stop(@PathVariable Long id) throws Exception {
        JobExecution exec = jobExplorer.getJobExecution(id);
        exec.setStatus(BatchStatus.STOPPING);
        return ResponseEntity.accepted().build();
    }
}
```

15.4 JobLauncher asynchrone

Par défaut le JobLauncher est synchrone. Pour un endpoint REST, il faut de l'asynchrone :

```
@Configuration
public class JobLauncherConfig {

    @Bean
    public JobLauncher asyncJobLauncher(JobRepository jobRepository) throws Exception {
        TaskExecutorJobLauncher launcher = new TaskExecutorJobLauncher();
        launcher.setJobRepository(jobRepository);
        launcher.setTaskExecutor(new SimpleAsyncTaskExecutor("batch-launcher-"));
        launcher.afterPropertiesSet();
        return launcher;
    }
}
```

16. Monitoring et observabilité

16.1 Actuator

Spring Boot Actuator expose des endpoints standards pour monitorer l'application :

```
management:
  endpoints:
    web:
      exposure:
        include: health,info,metrics,prometheus,batch
  endpoint:
    health:
      show-details: when_authorized
```

Endpoints utiles :

- `/actuator/health` : état général
- `/actuator/metrics` : métriques JVM, pools, HTTP
- `/actuator/prometheus` : format Prometheus pour Grafana

16.2 Métriques Spring Batch

Spring Batch publie automatiquement des métriques Micrometer. Visibles via Prometheus :

- `spring.batch.job` : durée, statut par job
- `spring.batch.step` : durée, statut, read/write/skip counts
- `spring.batch.chunk.write` : latence d'écriture par chunk

16.3 JobExecutionListener pour alertes

```
@Component
@RequiredArgsConstructor
@Slf4j
public class JobNotificationListener implements JobExecutionListener {

    private final NotificationService notifService;

    @Override
    public void beforeJob(JobExecution jobExecution) {
        log.info("=== Démarrage job {} (exec id={}) ===",
            jobExecution.getJobInstance().getJobName(),
            jobExecution.getId());
        notifService.envoyerInfo("Batch démarré",
            "Job " + jobExecution.getJobInstance().getJobName());
    }

    @Override
    public void afterJob(JobExecution jobExecution) {
        BatchStatus status = jobExecution.getStatus();
        long dureeMinutes = Duration.between(
            jobExecution.getStartTime(),
            jobExecution.getEndTime()).toMinutes();

        String resume = construireResume(jobExecution);

        if (status == BatchStatus.COMPLETED) {
            log.info("=== Job OK en {} min ===", dureeMinutes);
            notifService.envoyerSucces("Batch OK", resume);
        } else {
            log.error("=== Job KO : {} ===", status);
            notifService.envoyerAlerte("Batch KO", resume);
        }
    }

    private String construireResume(JobExecution exec) {
        StringBuilder sb = new StringBuilder();
        exec.getStepExecutions().forEach(step -> {
            sb.append(String.format("- %s : lu=%d écrit=%d skip=%d%n",
                step.getStepName(),
                step.getReadCount(),
                step.getWriteCount(),
                step.getSkipCount()));
        });
        return sb.toString();
    }
}
```

16.4 Logs structurés

En production, préférer le format JSON pour l'indexation par Elastic/Splunk :

```
<!-- logback-spring.xml -->
<configuration>
    <springProfile name="prod">
        <appender name="JSON" class="ch.qos.logback.core.ConsoleAppender">
            <encoder class="net.logstash.logback.encoder.LogstashEncoder">
                <customFields>{"app":"reprise-donnees"}</customFields>
            </encoder>
        </appender>
        <root level="INFO">
            <appender-ref ref="JSON"/>
        </root>
    </springProfile>
</configuration>
```

17. Gestion des secrets

17.1 Ne JAMAIS committer les secrets

> Les mots de passe BDD, tokens API, clés privées ne doivent jamais se trouver dans un fichier `application.yml` committé. Utiliser à la place : variables d'environnement, Vault, fichier externe.

17.2 Variables d'environnement

```
# application.yml
app:
  datasource:
    source:
      username: ${SOURCE_USER}
      password: ${SOURCE_PWD}
```

```
# Au lancement
export SOURCE_USER=legacy
export SOURCE_PWD=motdepasse
java -jar reprise.jar
```

17.3 HashiCorp Vault

```
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-vault-config</artifactId>
</dependency>
```

```
# bootstrap.yml
spring:
  cloud:
    vault:
      uri: https://vault.entreprise.fr:8200
      authentication: KUBERNETES
      kubernetes:
        role: reprise-donnees
      kv:
        enabled: true
        backend: secret
        application-name: reprise-donnees
```

17.4 Fichier externe

```
# Lancement avec un fichier de config externe non-committé
java -jar reprise.jar \
  --spring.config.additional-location=file:/etc/reprise/secrets.yml
```

18. Packaging et déploiement

18.1 Dockerfile multi-stage

```
FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /app
COPY pom.xml .
RUN mvn dependency:go-offline
COPY src ./src
RUN mvn package -DskipTests

FROM eclipse-temurin:21-jre-alpine
WORKDIR /app

# Utilisateur non-root pour sécurité
RUN addgroup -S batch && adduser -S batch -G batch
USER batch

COPY --from=build /app/target/reprise-donnees-*.jar app.jar

# JVM tuning pour batch
ENV JAVA_OPTS="-XX:+UseG1GC \
               -XX:MaxGCPauseMillis=200 \
               -XX:+HeapDumpOnOutOfMemoryError \
               -XX:HeapDumpPath=/tmp/heapdump.hprof"

ENTRYPOINT exec java $JAVA_OPTS -jar app.jar
```

18.2 Profils Spring

- `application.yml` : config commune
- `application-dev.yml` : base H2 en mémoire, logs DEBUG
- `application-recette.yml` : bases de recette, logs INFO
- `application-prod.yml` : bases de prod, logs JSON, pas de DEBUG

18.3 Helm chart (Kubernetes)

```
# values.yaml
replicaCount: 1                    # batch = 1 seul pod
image:
  repository: registry.entreprise.fr/reprise-donnees
  tag: "1.0.0"

resources:
  limits:
    memory: "4Gi"
    cpu: "2"
  requests:
    memory: "2Gi"
    cpu: "500m"

env:
  SPRING_PROFILES_ACTIVE: prod

secrets:
  vaultPath: secret/data/reprise-donnees/prod

# Job Kubernetes (pas Deployment) pour un batch ponctuel
job:
  enabled: true
  backoffLimit: 2
  activeDeadlineSeconds: 21600    # 6h max
```

19. Stratégie de rollback

19.1 Préparer le rollback avant le batch

> **Règle d'or :** prévoir le rollback *avant* de lancer le batch, pas pendant qu'il plante.

19.2 Snapshot BDD

```
# Avant le batch (PostgreSQL)
pg_dump -h new-db -U admin -d app -F c -f snapshot_avant_batch.dump

# Rollback si désastre
pg_restore -h new-db -U admin -d app --clean snapshot_avant_batch.dump
```

19.3 Flashback Oracle

```
-- Point de restauration avant batch
CREATE RESTORE POINT avant_reprise_20260420 GUARANTEE FLASHBACK DATABASE;

-- Rollback si désastre
FLASHBACK DATABASE TO RESTORE POINT avant_reprise_20260420;
```

19.4 Script de nettoyage ciblé

Si on ne veut nettoyer que les données du batch plutôt que tout restaurer :

```
-- rollback/vider_cibles.sql
-- Suppose un flag ou une trace de l'origine
BEGIN;

DELETE FROM ligne_commande
 WHERE commande_id IN (SELECT id FROM commande WHERE source_batch = 'REPRISE_V1');

DELETE FROM livraison
 WHERE commande_id IN (SELECT id FROM commande WHERE source_batch = 'REPRISE_V1');

DELETE FROM commande WHERE source_batch = 'REPRISE_V1';

DELETE FROM adresse
 WHERE id_client IN (SELECT id FROM client WHERE source_batch = 'REPRISE_V1');

DELETE FROM contact
 WHERE id_client IN (SELECT id FROM client WHERE source_batch = 'REPRISE_V1');

DELETE FROM client WHERE source_batch = 'REPRISE_V1';

-- Reset staging
TRUNCATE TABLE stg_client, stg_commande RESTART IDENTITY CASCADE;

COMMIT;
```

19.5 Plan de reprise documenté

Dans `docs/runbook-exploitation.md`, documenter :

1. Procédure de snapshot avant batch
2. Seuils qui déclenchent un rollback (ex : > 5% de rejets)
3. Commandes exactes à lancer
4. Qui valide le rollback (tech lead ? MOA ?)
5. Communication aux utilisateurs

20. Reprises incrémentales et bascule

20.1 Le problème

Si la source continue à vivre pendant qu'on prépare la migration, un batch unique ne suffit pas : il faut rattraper les changements entre deux exécutions.

20.2 Stratégie 1 : date de modification

```
-- Reprise initiale : tout
SELECT * FROM client_legacy;

-- Reprise delta : ceux modifiés depuis
SELECT * FROM client_legacy
 WHERE date_modification > :dernierBatch;
```

```
@Bean
@StepScope
public JdbcCursorItemReader<ClientLegacy> readerDelta(
        @Value("#{jobParameters['dernierBatch']}") String dernierBatch,
        @Qualifier("sourceDataSource") DataSource ds) {

    return new JdbcCursorItemReaderBuilder<ClientLegacy>()
        .name("readerDelta")
        .dataSource(ds)
        .sql("SELECT * FROM client_legacy WHERE date_modification > ?")
        .preparedStatementSetter(ps ->
            ps.setTimestamp(1, Timestamp.valueOf(dernierBatch)))
        .rowMapper(new ClientLegacyRowMapper())
        .build();
}
```

20.3 Stratégie 2 : hash de ligne (détection de modifs)

Si la source n'a pas de `date_modification` fiable, on compare les hash des lignes entre deux exécutions. Seules les lignes avec un hash différent sont retraitées.

20.4 Stratégie 3 : CDC (Change Data Capture)

Pour une synchronisation continue, Debezium capture les changements du log de la BDD source et les publie dans Kafka. Le batch devient alors un consumer Kafka. Au-delà du scope de ce document.

20.5 La bascule applicative

Scénario type :

1. **J-30** : Reprise initiale en recette, tests fonctionnels
2. **J-7** : Reprises delta quotidiennes pour rester synchrones
3. **J-1 soir** : Fermeture de l'ancien SI
4. **J-1 nuit** : Dernière reprise delta, réconciliation
5. **J0 matin** : Validation MOA, ouverture du nouveau SI
6. **J+7** : Période de double-check, rollback possible
7. **J+30** : Décommissionnement ancien SI

21. Stratégie de tests

21.1 Pyramide des tests pour un batch

Niveau	Quoi	Volume	Temps
Unitaires	Mappers, validators, services	70%	< 1s/test
Intégration	Steps isolés avec Testcontainers	20%	5-30s/test
Bout-en-bout	Job complet sur jeu représentatif	10%	Minutes
Volumétrie	Job sur volume prod en pré-prod	1 exécution	Heures

22. Tests unitaires du mapper

22.1 Cas à couvrir

- Cas nominal (toutes les données OK)
- Valeurs nulles tolérées (règles de défaut)
- Cas limites (chaînes vides, zéros, dates minimales)
- Cas de rejet (exceptions attendues)
- Éclatements 1→N (combien de lignes produites)

22.2 Exemple

```
@ExtendWith(MockitoExtension.class)
class CommandeMapperTest {

    @Mock ClientRepository clientRepository;
    @InjectMocks CommandeMapper mapper;

    @Test
    void doit_eclater_5_colonnes_produit_en_lignes() {
        // given
        StgCommande stg = new StgCommande();
        stg.setIdCmd(1L);
        stg.setIdClient(100L);
        stg.setMontantHt(new BigDecimal("100"));
        stg.setRefProd1("A"); stg.setQteProd1(2);
        stg.setRefProd2("B"); stg.setQteProd2(1);
        // prod 3, 4, 5 null

        when(clientRepository.findByIdExterne(100L))
            .thenReturn(Optional.of(new Client()));

        // when
        Commande cmd = mapper.toCommande(stg);

        // then
        assertThat(cmd.getLignes()).hasSize(2);
        assertThat(cmd.getLignes().get(0).getReferenceProduit()).isEqualTo("A");
        assertThat(cmd.getLignes().get(0).getQuantite()).isEqualTo(2);
    }

    @Test
    void doit_rejeter_si_client_inexistant() {
        // given
        StgCommande stg = new StgCommande();
        stg.setIdClient(999L);
        when(clientRepository.findByIdExterne(999L))
            .thenReturn(Optional.empty());

        // when / then
        assertThatThrownBy(() -> mapper.toCommande(stg))
            .isInstanceOf(DonneeInvalideException.class)
            .hasMessageContaining("Client inexistant");
    }

    @ParameterizedTest
    @CsvSource({
        ",'',               // tous nulls
        "'',0",             // ref vide
        "'ABC',0",          // qte zéro
        "'ABC',-1"          // qte négative
    })
    void ne_doit_pas_creer_ligne_si_donnee_invalide(String ref, Integer qte) {
        StgCommande stg = new StgCommande();
        stg.setIdClient(100L);
        stg.setRefProd1(ref); stg.setQteProd1(qte);

        when(clientRepository.findByIdExterne(any()))
            .thenReturn(Optional.of(new Client()));

        Commande cmd = mapper.toCommande(stg);
        assertThat(cmd.getLignes()).isEmpty();
    }
}
```

23. Tests d'intégration avec Testcontainers

23.1 Configuration de base

```
@Testcontainers
@SpringBootTest
@SpringBatchTest
class ChargementClientStepIT {

    @Container
    static PostgreSQLContainer<?> postgres =
        new PostgreSQLContainer<>("postgres:16-alpine")
            .withDatabaseName("test")
            .withUsername("test")
            .withPassword("test");

    @DynamicPropertySource
    static void configureProperties(DynamicPropertyRegistry registry) {
        registry.add("app.datasource.cible.jdbc-url", postgres::getJdbcUrl);
        registry.add("app.datasource.cible.username", postgres::getUsername);
        registry.add("app.datasource.cible.password", postgres::getPassword);
        // même chose pour source et batch selon le besoin
    }

    @Autowired JobLauncherTestUtils jobLauncherTestUtils;
    @Autowired JdbcTemplate jdbcTemplate;

    @BeforeEach
    void setUp() {
        // Jeu de test injecté en base
        jdbcTemplate.update("""
            INSERT INTO stg_client (id_legacy, nom, email, statut_traitement)
            VALUES (1, 'DUPONT', 'jean@test.fr', 'OK'),
                   (2, 'MARTIN', 'marie@test.fr', 'OK'),
                   (3, 'INVALID', 'pas-un-email', 'KO')
            """);
    }

    @Test
    void doit_charger_uniquement_les_lignes_OK() throws Exception {
        // when
        JobExecution exec = jobLauncherTestUtils.launchStep("chargementClientStep");

        // then
        assertThat(exec.getStatus()).isEqualTo(BatchStatus.COMPLETED);

        Long count = jdbcTemplate.queryForObject(
            "SELECT COUNT(*) FROM client", Long.class);
        assertThat(count).isEqualTo(2L);

        assertThat(jdbcTemplate.queryForObject(
            "SELECT nom FROM client WHERE id_externe = 1",
            String.class)).isEqualTo("DUPONT");
    }

    @Test
    void doit_etre_idempotent() throws Exception {
        // premier run
        jobLauncherTestUtils.launchStep("chargementClientStep");
        Long count1 = jdbcTemplate.queryForObject(
            "SELECT COUNT(*) FROM client", Long.class);

        // deuxième run
        jobLauncherTestUtils.launchStep("chargementClientStep");
        Long count2 = jdbcTemplate.queryForObject(
            "SELECT COUNT(*) FROM client", Long.class);

        assertThat(count2).isEqualTo(count1);  // pas de doublons
    }
}
```

24. Tests de volumétrie

24.1 Pourquoi

Un batch qui passe en 5 minutes sur 10 000 lignes peut mettre 20 heures sur 10 millions si le code a une complexité en O(n²) cachée (ex : un `findAll()` dans un processor). Les tests de volumétrie détectent ça avant la prod.

24.2 Plan de test type

Palier	Volume	Objectif
Smoke	100 lignes	Le batch démarre et termine
Fonctionnel	10 000 lignes	Toutes les règles métier sont vérifiées
Charge 10%	10% du volume prod	Détecter les régressions de perf
Charge 100%	Volume prod complet	Valider la fenêtre batch
Stress	120% du volume prod	Marge de croissance

24.3 Métriques à relever

- Durée totale du job et de chaque step
- Consommation mémoire max (via GC logs ou Prometheus)
- Consommation CPU
- Volume de logs produit
- Nombre de commits/rollbacks BDD
- Latence moyenne et p99 d'écriture par chunk

25. Aspects réglementaires (RGPD)

25.1 Obligations spécifiques à la RDD

Une reprise de données manipule souvent des données personnelles. Quelques obligations :

- **Base légale** documentée (continuité de service, intérêt légitime)
- **Minimisation** : ne migrer que les données strictement nécessaires
- **Purge** : les données non migrées doivent être supprimées de la source après rétention
- **Anonymisation des logs et rejets** : pas de noms/emails dans les traces accessibles à tous
- **Traçabilité** : qui a lancé quoi, quand, sur quelle volumétrie

25.2 Anonymisation dans les logs

```
public class LogMaskingUtils {
    public static String masquerEmail(String email) {
        if (email == null) return null;
        int at = email.indexOf('@');
        if (at < 2) return "***";
        return email.substring(0, 2) + "***" + email.substring(at);
    }

    // Utilisation dans les logs
    log.info("Rejet client id={} email={}",
        client.getIdExterne(),
        LogMaskingUtils.masquerEmail(client.getEmail()));
}
```

25.3 Environnements non-prod

Les environnements de recette/dev ne doivent pas contenir de vraies données perso. Prévoir un script d'anonymisation :

```
-- Anonymisation pour env de dev
UPDATE client SET
    email = 'user_' || id || '@test.local',
    nom = 'NOM_' || id,
    prenom = 'PRENOM_' || id,
    telephone = '0600000000',
    adresse = '1 rue de Test'
WHERE id IN (SELECT id FROM client);
```

26. Checklist complète pour un nouveau batch

Conception

- ☐ Document de mapping source→cible créé et validé par la MOA
- ☐ Règles de validation et rejets documentées
- ☐ Volumétrie estimée avec la MOA
- ☐ Fenêtre batch négociée avec l'exploitation
- ☐ Stratégie de rollback définie et validée

Infrastructure

- ☐ 3 datasources configurées : source, cible, batch
- ☐ Dimensionnement des pools de connexions
- ☐ Accès réseau entre les bases validé
- ☐ Secrets stockés dans Vault / variables d'env

Code

- ☐ Migration Flyway pour la table de staging
- ☐ Entité legacy en lecture seule
- ☐ Entité staging avec colonnes de pilotage
- ☐ Entités cibles avec contrainte UNIQUE sur clé métier
- ☐ Mapper avec tests unitaires > 80% de couverture
- ☐ Step d'extraction (source → staging)
- ☐ Step de validation (Tasklet SQL)
- ☐ Step de déduplication si applicable
- ☐ Steps de chargement (1 par table cible) dans le bon ordre FK
- ☐ SkipListener branché sur chaque step
- ☐ JobExecutionListener pour alertes
- ☐ Step de réconciliation finale
- ☐ UPSERT dans les writers pour idempotence

Tests

- ☐ Tests unitaires mapper
- ☐ Tests d'intégration avec Testcontainers
- ☐ Test bout-en-bout sur jeu représentatif
- ☐ Test d'idempotence (double exécution)
- ☐ Test de volumétrie en pré-prod

Exploitation

- ☐ Endpoint REST de lancement sécurisé (ou ordonnanceur)
- ☐ Dashboards Grafana/Kibana
- ☐ Alerting sur KO et écarts de réconciliation
- ☐ Runbook d'exploitation complet
- ☐ Procédure de restart documentée
- ☐ Procédure de rollback documentée et testée

Conformité

- ☐ Analyse d'impact RGPD si données personnelles
- ☐ Logs sans données sensibles en clair
- ☐ Scripts d'anonymisation pour non-prod
- ☐ Traçabilité des exécutions conservée

27. Erreurs fréquentes

Erreur	Conséquence	Solution
Lire/écrire source → cible sans staging	Rejeu impossible, données instables	Toujours passer par staging
Chunk size trop grand	OOM, rollback coûteux	Commencer à 500, mesurer
Pas de clé métier stable	Impossible de rejouer sans doublons	Ajouter `id_externe` UNIQUE
Ordre FK non respecté	Échec d'insertion à cause des FK	Parent avant enfants
Pas de `@StepScope` sur les readers paramétrés	Restart impossible, paramètres figés	Ajouter `@StepScope`
Logique métier dans le Reader/Writer	Code non testable	Logique dans Mapper + Service
Pas de SkipListener	Impossible de tracer les rejets	Ajouter log en base + CSV
Pas de réconciliation finale	Écarts silencieux source/cible	Tasklet de comptage + seuil
findAll() dans un processor	Complexité $O(n^2)$, batch interminable	Cache en début de step ou lookup ciblé
Pas de test de volumétrie	Découverte en prod d'un batch de 20h	Test sur volume prod en pré-prod
Secrets dans application.yml	Fuite lors d'un git clone	Variables d'env ou Vault
Pas de snapshot avant batch en prod	Impossible de rollback	pg_dump / flashback avant lancement

28. Pour aller plus loin

28.1 Ressources officielles

- **Spring Batch Reference** : docs.spring.io/spring-batch/reference/
- **Spring Boot Reference** : docs.spring.io/spring-boot/
- **Flyway Documentation** : documentation.red-gate.com/fd/
- **Testcontainers** : testcontainers.com

28.2 Sujets avancés non couverts

- **Remote chunking** : distribuer le traitement sur plusieurs JVM via JMS/AMQP
- **Remote partitioning** : partitionner sur plusieurs machines
- **Spring Cloud Data Flow** : orchestrer plusieurs jobs avec UI
- **Debezium + Kafka** : CDC pour synchronisation temps réel
- **Batch sur Kubernetes Jobs** : patterns cloud-native
- **Chaos engineering** : tester la résilience (Chaos Monkey)

28.3 Mot de la fin

> En reprise de données, ce qui coûte n'est pas le code mais **les cas particuliers imprévus** dans les données réelles. Prévoir du temps pour les itérations de recette avec la MOA, bâtir une mécanique de rejeu fiable dès le départ, et mesurer la volumétrie avant la prod.
>
> Un bon batch de reprise est un batch **rejouable, observable et réconcilié**. Le reste vient avec l'expérience.

— Fin du guide —